EXPERIMENT 11

Deploying a Zoho Creator Application from Development to Production Environment

LOW CODE APPLICATION • LABORATORY REPORT

Experiment aim

To outline a controlled deployment process that moves a Creator application from development to production while protecting data and reducing operational risk.

Application / case context

The supplied screenshots show the Gym Enrollment app running in a trial environment with an expiration notice. The screenshots do not establish that separate development and production environments have been configured. This experiment describes a deployment lifecycle and the checks needed before release.

Report structure: background, design, procedure, verification, results, conclusion and references. Any feature not confirmed by supplied evidence is identified as a proposed design or procedure.

24140107

1. Aim, Objectives and Environment Concepts

Development is where forms, fields, workflows, and reports are created and tested. Production is the environment used by real users and should contain only a reviewed, approved version of the application.

Objectives: distinguish development and production; prepare a release checklist; verify configuration and data dependencies; test permissions and workflows after deployment; and plan rollback or recovery.

Environment separation reduces the chance that experiments or debugging changes disrupt daily operations. Exact deployment and environment features depend on the Creator plan and current interface.

A production release should be treated as a controlled change. The release candidate needs a version identifier, a short change summary, a list of dependent connections and settings, and evidence that key workflows have been tested. A person other than the developer should review the most important checks where practical.

Environment-specific settings are a common source of deployment failures. A connection that was authorized during development may not be authorized in the target environment; an email sender or user permission may also differ. Verify these settings after promotion rather than assuming they transfer automatically.

Rollback planning must consider both application configuration and data created while a faulty release was active. Restoring an earlier version may not automatically undo records, emails, or external API actions. Define who can disable a workflow, how affected records will be identified, and how the team will communicate the correction.

A deployment is successful only when the live user journey works in the target environment. A published app that cannot send confirmation email, enforce validation, or protect member records is not release-ready. Keep the smoke-test results with the release notes.

The scope of this experiment is deliberately limited to a lab-scale application. It demonstrates the underlying concept and the verification method; it does not establish production readiness without the additional checks described in this report.

Objectives and scope

This experiment studies controlled promotion from development to production, including configuration review, smoke testing, and rollback planning.

24140172

2. Release Planning and Readiness

Before deployment, freeze the candidate version and list the forms, reports, workflows, custom functions, connections, and permissions that are part of the release. Identify dependencies such as email sender settings, API connections, and required user roles.

Check data quality. The current app screenshot contains age-11 records even though the intended age range is 14–75. Do not deploy until the validation defect is fixed or clearly contained and the invalid test records are handled.

Prepare test cases for form submission, report filtering, age boundaries, date validation, waiver acknowledgement, and confirmation email. Identify who approves the release and who will monitor the application after deployment.

A production release should be treated as a controlled change. The release candidate needs a version identifier, a short change summary, a list of dependent connections and settings, and evidence that key workflows have been tested. A person other than the developer should review the most important checks where practical.

Environment-specific settings are a common source of deployment failures. A connection that was authorized during development may not be authorized in the target environment; an email sender or user permission may also differ. Verify these settings after promotion rather than assuming they transfer automatically.

Rollback planning must consider both application configuration and data created while a faulty release was active. Restoring an earlier version may not automatically undo records, emails, or external API actions. Define who can disable a workflow, how affected records will be identified, and how the team will communicate the correction.

A deployment is successful only when the live user journey works in the target environment. A published app that cannot send confirmation email, enforce validation, or protect member records is not release-ready. Keep the smoke-test results with the release notes.

Deployment should include a communication plan: tell intended users when the application will be unavailable or changed, explain the new behaviour, and provide a contact for reporting problems. For a student lab, this may be a short release note rather than a formal production notice.

Record the exact version or change set that was tested. If a defect appears after release, the team should be able to identify which change introduced it and whether the issue can be corrected without affecting unrelated features.

Test the deployed application with the same roles that will use it. An administrator may be able to see all reports, while a front-desk user may have limited permissions. A deployment check performed only with an administrator account can miss a permission error that blocks normal users.

Record deployment verification as a checklist with date, tester, result, and issue notes. If a test fails, stop the release or apply the agreed workaround before announcing success. This gives the lab report a clear connection between deployment procedure and verified behaviour.

Design / configuration summary

Gate	Evidence required
Functional	Form and report work with valid data
Validation	Age 13/14/75/76 and date boundary tests pass
Automation	Confirmation email arrives once per valid submission
Security	Role permissions tested with separate users
Configuration	Connections and environment-specific settings checked
Recovery	Rollback or restoration plan documented

24140172

3. Deployment Procedure

Use Creator's supported application deployment or environment-promotion mechanism if available on the account. Review the target environment before publishing and confirm the destination is correct.

Move the reviewed app version, then verify environment-specific configuration such as connection authorization, email settings, user permissions, URLs, and any integration credentials. Do not assume every development connection is automatically valid in production.

Run smoke tests immediately after deployment: open the form, submit a safe test record, check the report, exercise filters, and confirm the workflow runs. Remove test records according to the agreed policy.

A production release should be treated as a controlled change. The release candidate needs a version identifier, a short change summary, a list of dependent connections and settings, and evidence that key workflows have been tested. A person other than the developer should review the most important checks where practical.

Environment-specific settings are a common source of deployment failures. A connection that was authorized during development may not be authorized in the target environment; an email sender or user permission may also differ. Verify these settings after promotion rather than assuming they transfer automatically.

Rollback planning must consider both application configuration and data created while a faulty release was active. Restoring an earlier version may not automatically undo records, emails, or external API actions. Define who can disable a workflow, how affected records will be identified, and how the team will communicate the correction.

A deployment is successful only when the live user journey works in the target environment. A published app that cannot send confirmation email, enforce validation, or protect member records is not release-ready. Keep the smoke-test results with the release notes.

Deployment should include a communication plan: tell intended users when the application will be unavailable or changed, explain the new behaviour, and provide a contact for reporting problems. For a student lab, this may be a short release note rather than a formal production notice.

Record the exact version or change set that was tested. If a defect appears after release, the team should be able to identify which change introduced it and whether the issue can be corrected without affecting unrelated features.

Test the deployed application with the same roles that will use it. An administrator may be able to see all reports, while a front-desk user may have limited permissions. A deployment check performed only with an administrator account can miss a permission error that blocks normal users.

Record deployment verification as a checklist with date, tester, result, and issue notes. If a test fails, stop the release or apply the agreed workaround before announcing success. This gives the lab report a clear connection between deployment procedure and verified behaviour.

Procedure and configuration checklist

- Prepare a release checklist and freeze the candidate version.
- Verify connections, permissions, email and other dependencies.
- Deploy through the supported Creator mechanism.
- Run smoke tests and confirm rollback steps.

24140172

4. Rollback, Monitoring and Governance

Define rollback criteria before release: repeated submission failures, incorrect validation, email failures, unauthorized access, or data corruption. Know how to restore a prior version or disable the faulty workflow using supported Creator controls.

Monitor errors, failed workflow actions, email delivery, and user feedback. Restrict production edits to authorized maintainers and record significant changes.

Back up or export data using supported methods before major changes when appropriate. A backup plan must be tested; the existence of an export is not proof that a full restoration will work.

A production release should be treated as a controlled change. The release candidate needs a version identifier, a short change summary, a list of dependent connections and settings, and evidence that key workflows have been tested. A person other than the developer should review the most important checks where practical.

Environment-specific settings are a common source of deployment failures. A connection that was authorized during development may not be authorized in the target environment; an email sender or user permission may also differ. Verify these settings after promotion rather than assuming they transfer automatically.

Rollback planning must consider both application configuration and data created while a faulty release was active. Restoring an earlier version may not automatically undo records, emails, or external API actions. Define who can disable a workflow, how affected records will be identified, and how the team will communicate the correction.

A deployment is successful only when the live user journey works in the target environment. A published app that cannot send confirmation email, enforce validation, or protect member records is not release-ready. Keep the smoke-test results with the release notes.

Deployment should include a communication plan: tell intended users when the application will be unavailable or changed, explain the new behaviour, and provide a contact for reporting problems. For a student lab, this may be a short release note rather than a formal production notice.

Record the exact version or change set that was tested. If a defect appears after release, the team should be able to identify which change introduced it and whether the issue can be corrected without affecting unrelated features.

Test the deployed application with the same roles that will use it. An administrator may be able to see all reports, while a front-desk user may have limited permissions. A deployment check performed only with an administrator account can miss a permission error that blocks normal users.

Record deployment verification as a checklist with date, tester, result, and issue notes. If a test fails, stop the release or apply the agreed workaround before announcing success. This gives the lab report a clear connection between deployment procedure and verified behaviour.

Verification checklist

- Verify app opens in the target environment.
- Test validation, report filters and email.
- Confirm recovery and data-correction steps.

24140172

5. Results and Conclusion

A deployment plan protects the application from untested changes and separates experimentation from operational use. The current screenshots show a working trial app but do not demonstrate a production deployment.

Conclusion: release readiness requires a stable data model, correct validation, working workflows, appropriate permissions, verified connections, and post-deployment smoke tests. Deployment should be treated as a controlled process rather than simply clicking publish.

A production release should be treated as a controlled change. The release candidate needs a version identifier, a short change summary, a list of dependent connections and settings, and evidence that key workflows have been tested. A person other than the developer should review the most important checks where practical.

Environment-specific settings are a common source of deployment failures. A connection that was authorized during development may not be authorized in the target environment; an email sender or user permission may also differ. Verify these settings after promotion rather than assuming they transfer automatically.

Rollback planning must consider both application configuration and data created while a faulty release was active. Restoring an earlier version may not automatically undo records, emails, or external API actions. Define who can disable a workflow, how affected records will be identified, and how the team will communicate the correction.

A deployment is successful only when the live user journey works in the target environment. A published app that cannot send confirmation email, enforce validation, or protect member records is not release-ready. Keep the smoke-test results with the release notes.

A good conclusion separates observed results from expected results. Report any failed test and the next corrective action plainly; a documented limitation is more useful than claiming a pass that the evidence does not support.

Learning outcomes

- Defined release gates and environment checks.
- Planned smoke tests and rollback.
- Recognized that deployment includes configuration and data.

Conclusion

Conclusion: release readiness requires a stable data model, correct validation, working workflows, appropriate permissions, verified connections, and post-deployment smoke tests. Deployment should be treated as a controlled process rather than simply clicking publish.

References

Zoho Creator Help: https://www.zoho.com/creator/help/

Zoho Creator environments overview: https://www.zoho.com/creator/

Zoho Creator application deployment guidance in official help documentation.

24140172

6. Release Checklist and Rollback Planning

A release candidate should include a version identifier, change summary, tested workflows, known limitations, and a list of configuration dependencies. The release owner should be able to explain what changed and how the change was verified.

Rollback is not simply returning the UI to a previous version if data has already changed. Decide whether records created during the faulty release need correction, export, or reconciliation. Never delete production records casually to clean up test data.

After deployment, monitor the first real submissions and check that the expected workflow actions run once. Confirm that permissions, email delivery, and report filters behave the same way in the target environment.

Deployment should include a communication plan: tell intended users when the application will be unavailable or changed, explain the new behaviour, and provide a contact for reporting problems. For a student lab, this may be a short release note rather than a formal production notice.

Record the exact version or change set that was tested. If a defect appears after release, the team should be able to identify which change introduced it and whether the issue can be corrected without affecting unrelated features.

Practical application to the lab

The supplied screenshots show the Gym Enrollment app running in a trial environment with an expiration notice. The screenshots do not establish that separate development and production environments have been configured. This experiment describes a deployment lifecycle and the checks needed before release.

Evaluation criteria

A satisfactory lab result should be reproducible, supported by a clear configuration or test record, and honest about limitations. Record the setup, test input, expected result, actual result, and evidence. If a feature was not enabled or tested in the current account, report that as a limitation and describe the procedure needed to complete it.

Checklist for submission

- Confirm the report matches the experiment title and stated objectives.
- Keep screenshots and configuration details aligned with what was actually performed.
- Separate verified outcomes from proposed implementation steps.
- Recheck the final PDF page layout before submission.

24140107

7. Deployment Test Plan and Final Recommendations

Post-deployment smoke tests should cover the highest-risk user journeys: valid enrollment, invalid age, past date, report filtering, and confirmation email. Test with a controlled record and remove or label it afterward.

Check that environment-specific connection permissions and sender configuration are valid in production. A successful development email does not prove that the production workflow has the same authorization or delivery behaviour.

Final recommendation: use a release checklist, a named owner, a rollback plan, and evidence of post-deployment tests.

Deployment should include a communication plan: tell intended users when the application will be unavailable or changed, explain the new behaviour, and provide a contact for reporting problems. For a student lab, this may be a short release note rather than a formal production notice.

Record the exact version or change set that was tested. If a defect appears after release, the team should be able to identify which change introduced it and whether the issue can be corrected without affecting unrelated features.

Practical application to the lab

The supplied screenshots show the Gym Enrollment app running in a trial environment with an expiration notice. The screenshots do not establish that separate development and production environments have been configured. This experiment describes a deployment lifecycle and the checks needed before release.

Evaluation criteria

A satisfactory lab result should be reproducible, supported by a clear configuration or test record, and honest about limitations. Record the setup, test input, expected result, actual result, and evidence. If a feature was not enabled or tested in the current account, report that as a limitation and describe the procedure needed to complete it.

Checklist for submission

- Confirm the report matches the experiment title and stated objectives.
- Keep screenshots and configuration details aligned with what was actually performed.
- Separate verified outcomes from proposed implementation steps.
- Recheck the final PDF page layout before submission.

24140172